Shri Shamrao Patil (Yadravkar) Educational & Charitable Trust's
Sharad Institute of Technology College of Engineering
YADRAV (Ichalkaranji) - 416 101. Tal. Shirol, Dist. Kolhapur (M.S.) India
+ Approved by AICTE, New Delhi + Recognized by Government of Maharashtra + Affiliated to DBATU, Lonere

+ **Lecture No. :** +

■ **Theory Content to be taught :** Unit 1.

1. Introduction to Data structure : Introduction
2.
3.
4.
5.

■ **Specific Objectives :**

1. Interpret the basics of data structure and
2. its application.
3.
4.
5.

■ **Theory Content :**

★ Introduction to Data :-

Data is a collection of raw facts, figures, observations, or symbols that by themselves may not have meaning.

After processing and organizing, data becomes useful information.

'Data is the raw material used by computers for processing and generating meaningful information.'

Examples :

Student roll numbers : 101, 102

Marks : 85, 90, 95

Names : abc, pqr

Temperature readings : 32°C, 34°C.

- Data Vs. Information.

Data	Information
1) Raw facts & figures.	Processed and meaningful data.
2) Unorganized	Organized.
3) Input to a system	Output from a system
4) May not have meaning	Useful for decision Making.

Example :-

- Data :- 70, 80, 90
- Information :- Average Marks = 80

Data Structure :-

A data structure is a way of organizing and storing data in a computer so that it can be accessed and used effectively.

The choice of a good data structure makes it possible to perform a variety of critical operations effectively.

A data structure is not only used for organising the data. It is also used for processing, retrieving, and storing data.

- Need of Data structure :-

Data structure are essential because they provide a systematic way to organize, manage, and store data in a computer so that it can be accessed and modified efficiently.

Core reasons why data structure are needed :-

a) Time Efficiency :-

proper structures significantly reduce the time needed to search, sort or modify data.

b) space Efficiency :-

They prevent memory wastage by choosing structural formats tailored to the exact data volume.

c) Data organization and Relationships :-

They reflect real-world logical ~~data~~ relationship

between pieces of information.

- Real world system implementations

specific structures are deployed accross various computer domains to resolve complex tasks :-

① operating system - Queues & linked lists.
- To manage CPU process scheduling and memory chunk allocations.

② Database Management - B-Trees & Hash Indexes.
- Powers rapid search indexing accross billions of record files.

③ Web Browsers - Tree formats.
- Organizes nested HTML elements and renders interface layouts.

④ GPS & Maps - Graphs.
- Models road intersections to calculate the fastest route paths.

⑤ Network Packets - stacks and Queues.
- Orders incoming data stream packets in the correct buffer sequence.

Classification of Data structures :-

Data structures can be categorized into two types :-

1. ~~Linear~~ Primitive data structure.
2. Non- Primitive data structure.

1) Primitive Data Structure :-

Primitive data structure are the most basic data structures available in programming language, such as int, float, char etc.

2) Non- Primitive Data Structure :-

Non- primitive data structures are complex data structures that are built using primitive data types, such as array, linked list, stack, queue, tree, graphs etc.

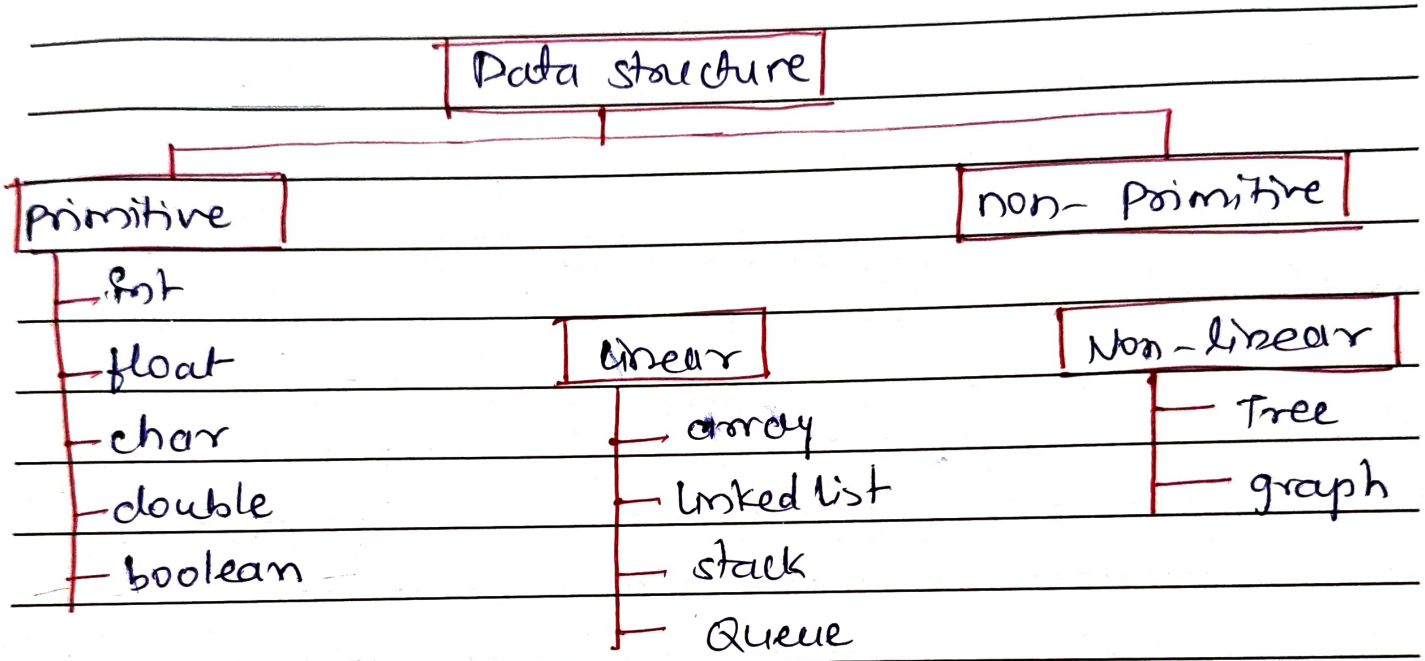
The choice of data structure for a particular task depends on the type and amount of data to be processed, the operations that need to be performed on the data and the efficiency requirements of the program.

A) Linear data structure :-

- Elements are arranged in one dimension, also known as linear dimension.
- Example - list, stack, queue etc.

B) Non- linear data structure :-

- Elements are arranged in one-to-many, many-to-one and many-to-many dimensions.
- Example - tree, graph, table etc.



+ **Lecture No. :** +

■ **Theory Content to be taught :**

1. Data structure operations: insertion, deletion,
2. traversal etc.
3. Algorithms.
4.
5.

■ **Specific Objectives :**

1. Interpret the basics of data structure and
2. it's application.
3.
4.
5.

■ **Theory Content :**

★ Data structure operations :-

The data which is stored in our data structures are processed by certain set of operations.

There are different types of operations that can be performed for the manipulation of data in every data structure.

① Traversing :-

This operations leads to accessing each record / node exactly once so that certain items in the record may be processed.

(This accessing and processing is sometimes called "visiting" the record).

② searching :-

Searching means to find a particular element in the given data structure. It is considered as successful when the required element is found.

Seerching is the operation which we can performed on data structures like array, linked-list, tree, graph etc.

③ Insertion :-

It is the operation which we apply on all the data structures. Insertion means to add an element in the given data structure. The operation of insertion is successfull when the required element is added to the required data structure.

It is unsuccessful in some cases when the size of the data structure is full and when there is no space in the data structure to add any additional element.

④ Deletion :-

It is the operation which we can apply on all the data structures. Deletion means to delete an element in the given data structure. The operation of deletion is successful when the required element is deleted from the data structure.

⑤ sorting :-

Arranging the data in some logical order (ascending or descending order). For example, in numerical increasing order or alphabetically.

⑥ Merging :-

Combining the data of two different sorted files into a single sorted file.

★ Algorithms :-

Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output.

Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language. From the data structure point of view, following are some important categories of algorithms -

a) search - Algorithm to search an element in a data structure.

b) sort - Algorithm to sort items in a certain order.

c) insert - Algorithm to insert item in a data structure.

d) update - Algorithm to update an existing item in a data structure

e) delete - Algorithm to delete an existing item from a data structure

• Characteristics of an Algorithm :-

Not all procedures can be called an algorithm. An algorithm should have the following characteristics :-

a) unambiguous / precise:

Algorithm should be clear and unambiguous/precise. Each of it's steps, and their inputs / outputs should be clear and must lead to only one meaning.

b) input :-

An algorithm should have 0 or more well defined inputs.

c) output :-

An algorithm should have 1 or more well defined outputs, and should match the desired output.

d) finiteness :-

Algorithm must terminate after a finite number of steps.

e) feasibility :-

should be feasible with the available resources.

f) Independent :-

An algorithm should have step-by-step directions, which should be independent

of any programming code.

+ **Lecture No. :** +

■ **Theory Content to be taught :**

1. Algorithms : Analysis of an algorithm,
2. Asymptotic Notations, Time space trade
3. off.
4.
5.

■ **Specific Objectives :**

1. Interpret the basics of data structure and
2. it's application.
3.
4.
5.

■ **Theory Content :**

★ Analysis of an algorithm :-

An algorithm is a finite sequence of well - defined instructions used to solve a problem.

When multiple algorithms solve the same problem, we need a method to determine which one is more efficient. This is achieved through algorithm analysis.

Objectives of Algorithm Analysis :-

1) Measure execution time
2) Measure memory consumption.
3) Compare different algorithms.
4) Select the most efficient solution for large inputs.

- Performance Analysis of algorithms :-

Performance analysis evaluates the resources required by an algorithm.
The two primary resources are -

a) Time Complexity :-

Time complexity is the amount of time required to execute an algorithm.

Time can means no. of comparisons between data types, inner loops are executed.

Time also can be calculated which language, hardware, program or compiler used.

Time Complexity = Compile time + Run / execution Time.

b) Space Complexity :-

Space complexity refers to the total memory required by an algorithm.

Components of space complexity :-

1) fixed part :

memory that does not depend on Input size.

Includes :

- ✓ program Instructions.
- ✓ Constants
- ✓ fixed variables.

2) Variable part :

Memory that changes with input size.

Includes :

- ✓ Dynamic allocation.
- ✓ Arrays
- ✓ Recursion stack.

Formula :

$$S(P) = C + SP(I).$$

Where -

- ✓ C = fixed part
- ✓ sp(I) = Variable part.

Example :

Algorithm : SUM (A, B).

step 1 : start

step 2 : C = A + B + 10

step 3 : stop.

Here we have three variables A, B and C and one constant.

Hence, $S(P) = 1 + 3$.

Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

- **Algorithm Case1 :-**

a) **Best Case Analysis :-**

Best case is the input to the algorithm which takes minimum time for execution of it.

Ex :- In search algorithm -

1 2 3 4 5 6 7 8

If key element is = 1.

The element is found at first position. It is the best case to search.

b) **Average Case Analysis :-**

In average case all possible sequence of size 'n' are input to algorithm and takes average time for execution.

Ex :- In search algorithm -

1 2 3 4 5 6 7 8

If the key element is = 4.

elements searched one by one and total no. of comparisons are computed.

c) Worst case analysis :-

Worst case is that, input to algorithm which takes maximum time for execution of it.

Ex :- in search algorithm -

1 2 3 4 5 6 7 8

If key element is = 8

The element is present as last element.

Here worst case is, search last element of the list.

★ Asymptotic Notations :-

1. Big 'O' notation :-

The notation O(n) is the way to express upper bound of an algorithm running time. It tells us the maximum time an algorithm can take. It is mainly used to describe the worst-case performance.

Example :

Consider, $T(n) = 3n + 5$.

for large values of n :

$T(n) = O(n)$.

because 'n' grows faster than the constant 5.

Example: Linear search.

```
for (i=0; i<n; i++)
{   if (a[i] == key)
        return i;
}.
```

In the worst case, the element is found at last position.
Total no. of comparisions = n.
Therefore O(n).

- Examples of Big O.

function	Big O
5	O(1)
$3n+2$	O(n)
$4n^2+3n$	$O(n^2)$
n^3+2n	$O(n^3)$.

2. Omega Notation (Ω):

Omega notation represents the lower bound of an algorithm. It tells us the minimum time required by the algorithm. It is generally best case.

Example: Linear search, array = 10 20 30 4
searching for 10. The element is found in the first comparision.
Total no. of comparision = 1.
Therefore - $\Omega(1)$.

Shri Shamrao Patil (Yadravkar) Educational & Charitable Trust's
Sharad Institute of Technology College of Engineering
YADRAV (Ichalkaranji) - 416 101. Tal. Shirol, Dist. Kolhapur (M.S.) India
+ Approved by AICTE, New Delhi + Recognized by Government of Maharashtra + Affiliated to DBATU, Lonere

+ **Lecture No. :** +

■ **Theory Content to be taught :**

1. Asymptotic Notations.
2. Time space trade off.
3.
4.
5.

■ **Specific Objectives :**

1. Interpret the basic of data structure and
2. its application.
3.
4.
5.

■ **Theory Content :**

3. Theta Notation (Θ) :-

Theta notation gives both the upper bound and lower bound. It represents the exact growth rate of an algorithm.

Example - $T(n) = 5n^2 + 4n + 2$

Upper bound = $O(n^2)$

lower bound = $O(n^2)$.

Therefore - $\theta(n^2)$.

- Comparison of all notations :-

Notation	Meaning	Represents.
$O(g(n))$	Upper bound	Worst case
$\Omega(g(n))$	lower bound	Best case
$\theta(g(n))$	Exact bound	Average Case.

ⓐ Time - Space Tradeoff :-

An algorithm uses two important resources-

1) Time :- How long algorithm takes to execute.

2) space :- How much memory the algorithm uses.

Sometimes, we can reduce the execution time by using extra memory. Similarly, we can reduce memory usage but it may increase execution time.

This relationship is called the Time-Space Tradeoff.

- principle of Time - Space Tradeoff :-

✓ more space - Less Time.

If more memory is available, we can store intermediate results and avoid repeated calculations.

✓ Less space - More Time :

If memory is limited, calculations may need to be repeated, increasing execution time.

Example :- searching an element.

List = 10 20 30 40 50 60

Method 1 :- Linear search.

To find 60, we compare each element one-by-one.

Time Complexity = O(n).

Space Complexity = O(1).

Uses less memory but takes more time.

Method 2: Hash Table.

Store elements in hash table.

Searching Takes = O(1).

Space Complexity = O(n).

Uses more memory but takes less time.

Tradeoff - Extra memory is used to achieve faster searching.

+ **Lecture No. :** +

■ **Theory Content to be taught :**

1. searching : Linear search.
2.
3.
4.
5.

■ **Specific Objectives :**

1. Interpret the basics of data structure and
2. it's application.
3.
4.
5.

■ **Theory Content :**

* **Searching :-**

Computer systems are often used to store large amount of data from which individual records must be retrieved according to the search criteria.

Searching refers to the operation of finding the position of the element to be searched in a given list. If the

element does appear in that ~~test~~ list, then the location of that element is recorded and searching is said to be sucessful. Otherwise the search is said to be unsuccessful.

There are two popular searching algorithms. They are —

(a) Linear search / sequential search.

(b) Binary search.

1) Linear search :-

Linear search is the most fundamental and important searching algorithm. It is simple to understand and implement. Linear search which is also known as sequential search, means starting at the beginning of the list and checking each item in turn until either the desired item is found or the end of the list is reached indicating the item is not found. If the item to be searched is found in the list more than once, the number of occurrence of the item in the list is assumed to be one.

As each element is checked with search element, the process of searching requires more time.

* Example of Linear search :-

Consider the following array -

array = [10, 20, 30, 40, 50].

Suppose we want to search for :

key = 30.

Step 1: Compare key element with first element

30 == 10 ? No.

Move to the next element.

(7) Algorithm :-

step 1 : start.
step 2 : Read the array elements.
step 3 : Read the ~~array~~ element to be searched (key).
step 4 : set i = 0;
step 5 :
Compare a[i] = key.
~~display~~

step 6 : if a[i] = key.
display 'element found at pos+1'.
go to step 9.
step 7 : Otherwise increment i and repeat step 5 until all elements are checked.

step 8 : if key is not found,
display 'element not found'.

step 9 : stop.

Step 2: Compare key with second element.
30 == 20 ? No.
Move to next element.

Step 3: Compare key with third element.
30 == 30 ? yes.
Element found.

Result :
Element is found at position 3.

- Time Complexity Analysis :-
The efficiency of linear search depends on the position of the element.

a) Best case :-
The element is found at the first position.

Example : a = [10, 20, 30, 40, 50].
key = 10.
Only one comparision is needed.

Complexity = O(1).

b) Worst case :-

The element is found at the last position or does not exist.

Example:

a = [10, 20, 30, 40, 50].

key = 50. or key = 60.

All elements are checked.

Complexity = O(n).

c) Average case :-

the element may occur anywhere in the array.

Average no. of Comparisons : $(n+1)/2$.

Complexity = O(n)

★ Advantages of Linear search :-

1) Easy to understand.
2) Easy to implement.
3) Works on both sorted and unsorted array.
4) Requires no extra memory.
5) Suitable for small datasets.

★ Disadvantages of Linear Search :-

1) slow for large datasets.
2) Requires many comparisons.
3) searching time increases as data size increases.

'C' program for linear search :-

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a[100], n, key;
    int i, found=0;

    printf("enter no. of elements");
    scanf("%d", &n);

    printf("enter array elements");
    for(i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }
printf("enter the element to be searche
scanf("%d", &key);

for(i=0; i<n; i++)
{
    if(a[i]==key)
    { printf("element is found at pos=
                                i+1);
      found=1;
      break;
    }
}
```

Questions From DBATU Examination :

```
if (found==0)
{
    printf("element not found");
}
getch();
}
```

Question Bank :

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Searching : Binary search
2.
3.
4.
5.

■ **Specific Objectives :**

1. Interpret the basics of data structure and
2. it's application.
3.
4.
5.

■ **Theory Content :**

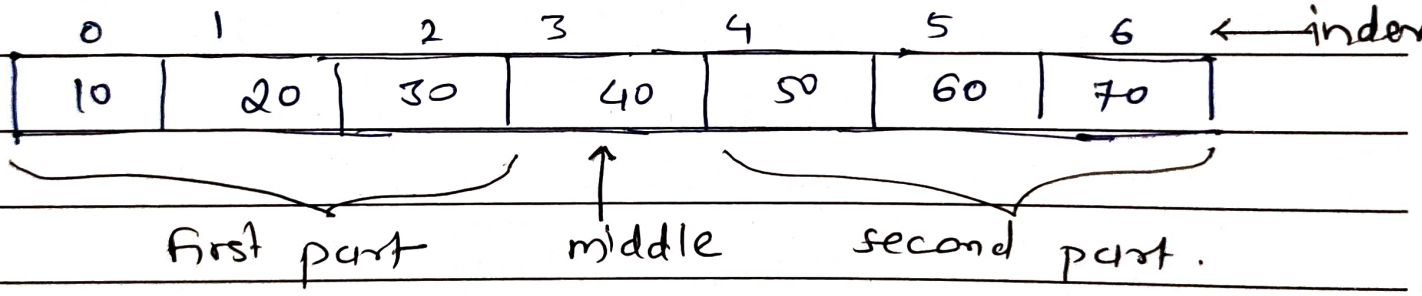
2) Binary search :-

Binary search uses divide and conquer approach to find the position of a specified value within a sorted list. (Elements requires in ascending order).

To search for an element in the list, the binary search algorithm divides the list and locates the middle element of the list.

It is then compaired with the search element. If the search element is less than the middle element, the first part of the list is searched else the second part of the list is searched.

The algorithm again reduces the list into two parts, locates the middle element and compares with the search element. If the search element is less than the middle element, the first part of the list is searched. This process continues until the search element is equal to the middle element or the list consist of only one element that is not equal to the search element.



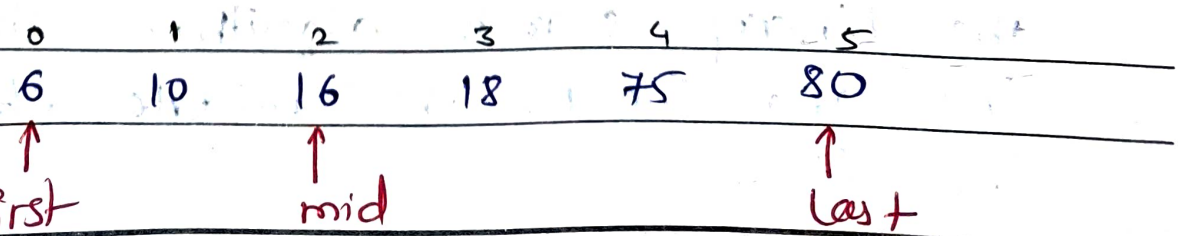
Example :-

Consider the following list -

10, 6, 16, 18, 80, 75.

Pre-condition for the Binary search is array elements must be in asceding order.

sorted list - 6, 10, 16, 18, 75, 80.



consider, search item = 18, will be compared with the mid element 16.

Here, 18 != 16, 18 is greater than 16, so search in the right sublist only. (18,75,80

0	1	2	3	4	5
6	10	16	18	75	80
			↑	↑	↑
			first	mid	last

search item = 18, will be compared with mid element 75. Here, 18 != 75, 18 is smaller than 75. So search in the left sublist only (18

0	1	2	3	4	5
6	10	16	18	75	80
			↑	↑	↑
			first	mid	last

search item = 18, will be compared with mid element 18. Here, 18 == 18. print position and return.

- **Algorithm :-**

Let arr[max] is an array and 'n' is the size of an array. 'key' is an element to be searched.

Step 1 : start.

step 2 : set first = 0

last = n-1

mid = (first + last) / 2.

step 3 : Repeat step 4 and 5
while first <= last && arr[mid] != key.

step 4 : if key > arr[mid] then
set first = mid + 1;
else
if key == arr[mid] then
print "key element found".
else
set last = mid - 1.

step 5 :
set mid = (first + last) / 2.

step 6 : if first > last then
print "key not found".

step 7 : stop.

- C program to implement Binary search.

```
#include <stdio.h>
void main()
{
  int first, last, mid, n, i, key, a[100];
  printf("enter the size of array");
  scanf("%d", &n);
  printf("enter %d elements in ascending order");
  for(i=0; i<n; i++)
    { scanf("%d", &a[i]);
    }
  printf("enter the value to be searched");
  scanf("%d", &key);
  first = 0;
  last = n-1;
  mid = (first + last)/2;
  while (first <= last)
  { if (a[mid] < key)
      first = mid + 1;
    else if (a[mid] == key)
    {
      printf("element found at pos %d", mid
      break;
    }
    else
      last = mid - 1;
      mid = (first + last)/2;
  }
  if (first > last)
    printf("element not found");
  getch();
}.
```

@ Time Complexity :-

a) Best Case :-

Condition : element is found at middle in first attempt.

Complexity = $O(1)$.

Because, only one comparision required.

b) Worst Case :-

Condition :- Element is found after multiple divisions or not present.

Complexity = $O(\log n)$.

Explaination :- Array is divided into half at every step.

Example :- for $n = 16 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$.

Only 4 steps needed.

c) Average Case :-

Complexity = $O(\log n)$.

On average, search space keeps reducing by half.

★ Advantages of Binary search :-

1. very fast searching technique.
2. Works well for large datasets.
3. Reduces search space quickly.
4. Requires fewer comparisions than linear search.

★ Disadvantages of Binary search :-

1) Works only on sorted arrays.
2) More complex than linear search.
3) sorting is required before searching.

◉ Linear search Vs. Binary search.

Feature	Linear search	Binary search.
Data Type	sorted / unsorted	only sorted.
Method	sequential	Divide & conquer.
Time Complexity	O(n)	O(log n).
speed	slow	Fast
Best for	Small Data	Large sorted data.

Shri Shamrao Patil (Yadravkar) Educational & Charitable Trust's
Sharad Institute of Technology College of Engineering
YADRAV (Ichalkaranji) - 416 101. Tal. Shirol, Dist. Kolhapur (M.S.) India
+ Approved by AICTE, New Delhi + Recognized by Government of Maharashtra + Affiliated to DBATU, Lonere

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Design a 'c' program to traverse array
2. for inserting and deleting an element.
3.
4.
5.

■ **Specific Objectives :**

1. Interpret the basics of data structure and
2. it's application.
3.
4.
5.

■ **Theory Content :**

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a[100], n;
    int pos, element;
    clrscr();
```

```
// Input array.

printf ("Enter no. of elements");
scanf ("%d", &n);

printf ("enter array elements");
for(i=0; i<n ; i++)
  {
     scanf ("%d", &a[i]);
  }
```

// Traversal.

```
printf (" Array elements are —");
for(i=0; i<n ; i++)
  {
     printf ("%d \t", a[i]);
  }
```

// insertion.

```
printf ("enter the position for insertion
          (1 to %d) : ", n+1);
scanf ("%d", &pos);

printf ("enter element to insert ");
scanf ("%d", &element);
```

```
for (i= n;  i>= pos ; i--)
 {
      a[i ] = a[i-1];
 }
 a[pos-1] =  element ;
 n++;

 printf ("\n Array after insertion ");
 for (i=0; i<n; i++)
 {
      printf ("%d\t", a[i]);
 }

// Deletion

printf ("enter the position to delete (1 to %d)", n);
scanf ("%d", &pos);

for (i= pos-1 ; i< n-1 ; i++)
 {
      a[i] = a[i+1];
 }
 n--;

 printf (" Array after deletion ");
 for (i=0; i<n; i++)
 {
      printf ("%d\t", a[i]);
 }
 getch();
}
```